

A Fixed Two-Sort $\Pi/\Sigma/\text{Id}$ Calculus

Regular Typing, Normalization, and Historical MeTTa Experiments

Zar Oruži*

September 2026

Abstract

We describe the fixed two-sort dependent-calculus fragment previously presented as “MeTTa-Pure.” Its mathematical content is a distinguished ground type $\mathbf{u0}$, an untyped formation marker $\mathbf{u1}$ with $\mathbf{u0} : \mathbf{u1}$, dependent products and sums, and identity formation with reflexivity. It has neither an identity eliminator nor a cumulative universe hierarchy. Its intrinsically scoped syntax supports two different typing relations: an older permissive relation and a separately defined, presupposition-closed regular judgment. Their strict separation is proved.

For the regular, declaration-free judgment, the Lean development proves strong normalization, supplies exact conversion by computed normal forms, and proves the Boolean checker’s equivalence with an independently stated bidirectional calculus. A syntactic category with families and its source-retaining presheaf interpretation preserve actual substitution and terms, without asserting initiality or identifying beta conversion with equality of semantic sections.

MeTTa-specific Pattern presentations, certificate services, and runtime bridges are experiments and adapters around this calculus. They are not a definition or adoption of Prime. We distinguish their verified results from the separate cumulative-tower candidate and from unproved transfers between these developments.

Contents

1	Scope and mathematical identity	2
2	The fixed two-sort fragment	2
2.1	Scoped syntax and reduction	2
2.2	Ground type and formation marker	3
3	Two typing relations, not two names for one judgment	4
3.1	The older permissive relation	4
3.2	The separate regular boundary	4
3.3	What the least-closure theorem says	4
4	Normalization and checking	5
4.1	Complete development is not a normalizer	5
4.2	Strong normalization of the regular judgment	5
4.3	Exact conversion on a proved domain	5
4.4	Bidirectional checking: the exact completeness claim	6

*“Oruži” denotes AI-assisted collaboration using Claude Code (Anthropic), ChatGPT, and Codex (OpenAI).

5	Categorical structure with its equality boundary	6
6	MeTTa presentations and certificate experiments	7
6.1	Ownership of the outer presentation	7
6.2	Regular Pattern elaboration	7
6.3	Historical closed certificates	7
7	Declaration and family experiments	8
8	The verified runtime frontier	8
9	Relation to a candidate Prime theory	9
10	Source and claim index	10
11	Conclusion	10

1 Scope and mathematical identity

The subject of this paper is the concrete calculus `TwoSortPiSigmaId`, not a completed native type theory for the MeTTa family. The historical filename `PureMeTTa.tex` is retained for existing references; it does not identify the calculus with Prime or select it as a production proof target.

Three distinctions govern the statements below.

1. Scoped syntax, permissive typing, and regular typing are different objects. A theorem about one does not silently strengthen the others.
2. Intrinsic mathematics belongs to `Mettapedia/TypeTheory/Calculi/TwoSortPiSigmaId/`. MeTTa-specific syntax and integration belong to `Mettapedia/Languages/MeTTa/Experimental/TwoSortPiSigmaId/`. Directory ownership follows the mathematical object being defined.
3. The separate cumulative-tower presentation investigated by Prime has its own syntax, judgments, and theorem hypotheses. Reuse requires a stated translation or interpretation and the corresponding preservation result.

This separation preserves useful work. It does not classify the whole development as a failed theory: substitution, confluence, the regular normalization theorem, exact checking boundaries, and the categorical constructions remain concrete results. It does remove the inference that having these results makes this particular calculus the intended Prime language.

All source references below are repository-relative Lean source paths. Within sections on the intrinsic calculus, a short filename is relative to `Mettapedia/TypeTheory/Calculi/TwoSortPiSigmaId/`; the MeTTa adapter root is always identified separately.

2 The fixed two-sort fragment

2.1 Scoped syntax and reduction

`Syntax.lean` defines `ScopedTermn`, indexed by de Bruijn depth. The index guarantees scope, not typing. The raw grammar is

$$\begin{aligned}
t, A, B ::= & x \mid c \mid u0 \mid u1 \mid \Pi(A, B) \mid \Sigma(A, B) \mid \text{ld}(A, t, t) \\
& \mid \lambda t \mid tt \mid (t, t) \mid \text{fst } t \mid \text{snd } t \mid \text{refl}(t).
\end{aligned}$$

The bodies of Π , Σ , and λ have one additional scoped variable. Global constants c exist in the raw grammar for declaration-aware experiments; they are excluded by the regular judgment in a regular context.

`Renaming.lean`, `Substitution.lean`, and `Context.lean` provide renaming, simultaneous substitution, and dependent telescopes. In particular, `inst0aB` substitutes for the newest bound variable; it is not an application constructor.

`Reduction.lean` defines the compatible closure of

$$(\lambda b) a \longrightarrow b[a/0], \quad \text{fst}(a, b) \longrightarrow a, \quad \text{snd}(a, b) \longrightarrow b.$$

There is no identity-elimination computation rule in this grammar. Parallel reduction and complete development establish confluence: `redStar_confluence` and `church_rosser_conv` occur in `Confluence.lean`. These are statements about the specified reduction relation, not about unrestricted runtime rewriting.

2.2 Ground type and formation marker

The two distinguished heads have the rule

$$\Gamma \vdash_{\text{reg}} \mathbf{u0} : \mathbf{u1}.$$

Here $\mathbf{u0}$ is a ground type and $\mathbf{u1}$ is its formation marker. The regular calculus has no rule assigning a type to $\mathbf{u1}$. Product and sum formation use the same fixed marker at domain, codomain, and result:

$$\frac{\Gamma \vdash_{\text{reg}} A : \mathbf{u1} \quad \Gamma, x : A \vdash_{\text{reg}} B : \mathbf{u1}}{\Gamma \vdash_{\text{reg}} \Pi(x : A).B : \mathbf{u1}}, \quad \frac{\Gamma \vdash_{\text{reg}} A : \mathbf{u1} \quad \Gamma, x : A \vdash_{\text{reg}} B : \mathbf{u1}}{\Gamma \vdash_{\text{reg}} \Sigma(x : A).B : \mathbf{u1}}.$$

The regular-context premise is implicit in this displayed notation. Identity formation and reflexivity are

$$\frac{\Gamma \vdash_{\text{reg}} A : \mathbf{u1} \quad \Gamma \vdash_{\text{reg}} a : A \quad \Gamma \vdash_{\text{reg}} b : A}{\Gamma \vdash_{\text{reg}} \text{ld}(A, a, b) : \mathbf{u1}}, \quad \frac{\Gamma \vdash_{\text{reg}} A : \mathbf{u1} \quad \Gamma \vdash_{\text{reg}} a : A}{\Gamma \vdash_{\text{reg}} \text{refl}(a) : \text{ld}(A, a, a)}.$$

These rules do not make $\mathbf{u0}$ a Russell universe of types. In particular, the formation premise is $A : \mathbf{u1}$, not $A : \mathbf{u0}$. Nor is $\mathbf{u1}$ the first member of a larger hierarchy: there are no further sort constructors, cumulativity rules, or universe-polymorphic quantification rules in this fragment. The names historically printed as “Type0” and “Type1” must not supply rules absent from the calculus.

Positive examples include

$$\emptyset \vdash_{\text{reg}} \lambda x.x : \Pi(x : \mathbf{u0}).\mathbf{u0}$$

and, in the regular context $x : \mathbf{u0}$,

$$x : \mathbf{u0} \vdash_{\text{reg}} \text{refl}(x) : \text{ld}(\mathbf{u0}, x, x).$$

The latter is a genuine term-dependent identity family. Negative examples are $\mathbf{u1} : \mathbf{u1}$ and a regular product whose domain is $\mathbf{u1}$. The absence of J means these identity formers do not constitute the full identity-type rules of intensional Martin-Löf type theory.

3 Two typing relations, not two names for one judgment

3.1 The older permissive relation

`Permissive/Typing.lean` defines `HasType`. Its variable rule accepts a raw telescope without a context-formation derivation. Its lambda rule lacks explicit domain and codomain formation; its conversion rule does not require formation of the target type.

For example, the source theorem `raw_allows_untyped_lambda_domain` proves

$$\emptyset \vdash_{\text{perm}} \lambda x.x : \Pi(x : \mathbf{u1}).\mathbf{u1}.$$

This is a derivation under the stated permissive rules, not a proof that the displayed function type is regularly formed. The permissive subject-reduction result remains a theorem about that relation in `Permissive/SubjectReduction.lean`. It does not supply the missing presuppositions.

3.2 The separate regular boundary

`Regular/Typing.lean` defines the following objects.

- `RegularHasType` retains formation premises for the dependent constructors and distinguishes conversion to a formed ordinary type from conversion to the special marker.
- `RegularCtx` requires every telescope extension to have a derivation of formation in the preceding context.
- `RegularJudgment` packages both a `RegularCtx` and a `RegularHasType` derivation.
- `ConstantFreeConv` is conversion whose derivation remains in the declaration-free fragment, including its reflexivity and intermediate terms.

The presupposition theorem `RegularJudgment.type_presupposed` in `Regular/Structural.lean` separates the exceptional result `u1` from ordinary result types that themselves have a formation derivation.

Proposition 3.1 (Strict separation). *Every regular typing derivation forgets to a permissive one, but the converse fails. In particular, the permissive identity at $\Pi(x : \mathbf{u1}).\mathbf{u1}$ has no regular typing derivation.*

The rejection theorem `regular_rejects_untyped_lambda_domain` is in `Regular/Typing.lean`. The comparisons `regular_strictly_below_raw` and `RegularJudgment.toHasType` are in the separate adapter `Mettapedia/Languages/MeTTa/Experimental/TwoSortPiSigmaId/Adapters/PresentationBoundary.lean`.

Context formation is independently important. In that same adapter, `raw_types_variable_in_top_sort_context` gives the permissive variable judgment in the raw telescope $x : \mathbf{u1}$; `rawTopSortContext_not_regular` and `no_regular_judgment_in_top_sort_context` reject that telescope at the regular boundary. Likewise, `regular_rejects_conversion_to_unformed_type` prevents conversion from admitting an unformed target merely because it beta-reduces to the marker.

3.3 What the least-closure theorem says

`RegularHasType.least` proves inclusion in every relation closed under the explicitly listed regular rules. This is the ordinary induction principle for the finite rule-generated judgment, presented as a least rule-closure result.

It is not a classifying initiality theorem for categories with families, an adjunction between language presentations, or a theorem identifying a universal Prime type theory. Those require different objects and universal properties. A proof of least closure is retained with its actual scope.

4 Normalization and checking

4.1 Complete development is not a normalizer

The function `cdev` contracts one complete development. The raw confluence development proves that a term reduces to, and is convertible with, that development. It does not prove that one application of `cdev` produces a normal form.

This distinction is witnessed inside the regular-support code: `Regular/DefEq.lean` defines the earlier one-development comparison `regularDefEq?` and proves `regularDefEq_not_complete`. Convertible, constant-free terms can have unequal one-pass developments.

Accordingly, the historical MeTTa service `Services/CompleteDevelopment.lean` packages a reduction and conversion to `cdev` in `CanonicalClosedTwoSortTerm`. Its `defEqClosed?` produces a sound conversion witness when the developments agree. The record does not contain a normal-form proof, and a failed one-pass comparison is not evidence of non-convertibility. Here and below, `Services/` is under the MeTTa experimental root stated in Section 1.

4.2 Strong normalization of the regular judgment

Theorem 4.1 (Regular strong normalization). *For every regular context and every `RegularJudgment` of a term at a type, the subject is accessible under the reverse reduction relation.*

The precise statement is `RegularJudgment.subject_accessible` in `Regular/Fundamental.lean`. Its proof uses the semantic realization of regular contexts, `RegularCtx.semantic_exists`, and the simultaneous formation-and-membership theorem `RegularHasType.fundamental`.

The theorem covers open regular judgments, not just closed terms. It does not cover arbitrary raw syntax or the declaration-aware extensions. The positive control is `regular_identity_accessible`. The negative control is `regular_omega_has_no_judgment`: the self-applicative term

$$\Omega = (\lambda x.x x)(\lambda x.x x)$$

has no regular judgment at any type. The raw term reduces to itself, as proved by `regularOmega_reduces_to_self` in `Regular/NormalizationBoundary.lean`.

4.3 Exact conversion on a proved domain

`Regular/NormalizationAlgorithm.lean` defines a certified reducer and instantiates the generic accessibility-recursive algorithm in `Mettapedia/Logic/Relation/Normalization.lean`. The specialized entry point is `normalizeAccessible`; its result carries an actual normal form and a reduction to that form. The generic construction depends only on a reduction relation, a complete step selector and accessibility, not on this calculus or a language encoding. It also proves normal-form uniqueness and strategy independence under confluence. The intrinsic simply typed fragment uses the same algorithm; separate numeric strategies and counterexamples in `Mettapedia/Logic/Relation/NormalizationExamples.lean` test its boundaries. `Regular/Normalization.lean` then constructs `regularNormalizationSpecification`: the fundamental theorem supplies coverage of regular subjects, and type presupposition supplies coverage of their types. The untyped marker `u1` is included separately as a normal term; normalizing it does not assign it a type.

For inputs in this certified, declaration-free normalization domain, `regularDecidedConversion_correct` states

$$\text{decideConversion}(s, t) = \text{true} \iff \text{ConstantFreeConv}(s, t).$$

The decision is comparison of computed normal forms (`regularDecidedConversion_is_direct`). Termination evidence is propositional and erased on execution. This is stronger than the earlier one-development comparison, but remains a domain-specific theorem.

4.4 Bidirectional checking: the exact completeness claim

`Regular/Bidirectional.lean` implements synthesis and checking using exact regular conversion. `Regular/BidirectionalCompleteness.lean` defines an independent direction-indexed relation `RegularBidirectional` and its public query predicate `RegularPublicChecks`.

The central executable equivalence is exactly

$$\begin{aligned} \text{regularCheckBool } \Gamma \ t \ A &= \text{true} \\ \iff \text{RegularPublicChecks } \Gamma \ t \ A, \end{aligned}$$

with a regular-context witness understood. The theorem is `regularCheckBool_correct`; `RegularPublicChecks.sound` maps the right-hand side to `RegularHasType`. Together with the supplied regular context, this yields a `RegularJudgment`.

The completeness theorem is against the independent bidirectional specification. It should not be restated as completeness for every declarative `RegularJudgment` without a separate theorem connecting that larger claim to the bidirectional rules. The direct `regularTypingDecisionKernel` packages this exact public-query meaning; it does not accept a user-supplied proof certificate as a substitute for computing its Boolean result.

Executable positive and negative controls include the identity at $\Pi(x : u0).u0$ (`checkRegular_identity_accepts`) and $u0 : u1$ (`regularTypingDecisionKernel_accepts_positive`). The judgment $u1 : u1$ is rejected by `regularTypingDecisionKernel_rejects_negative`. Regular subject reduction, used when checking inspects normalized type heads, is proved separately in `Regular/SubjectReduction.lean`.

5 Categorical structure with its equality boundary

`Regular/SyntacticCwf.lean` constructs a category with families from the actual regular syntax:

- contexts are regular telescopes;
- types and terms are scoped expressions carrying their regular derivations;
- arrows are well-typed simultaneous substitutions;
- comprehension is telescope extension.

The definitions `cwf` and `cwfWithTerminal` prove the structural laws and terminality of the empty context. Dependent product, sum, identity formation, and their term operations use the existing constructors. The substitution laws are not supplied by an identity or constant substitute for the intended construction.

Equality in this CwF is equality of raw syntax. Beta computation is instead expressed by `apply_lambda_converts`, a `ConstantFreeConv` statement. Thus this construction must not be advertised as a conversion-quotiented initial model.

`Regular/PresheafSemantics.lean` applies the generic `Mettapedia/TypeTheory/CwfYoneda.lean` machinery to this CwF. `representedTermEquiv` identifies source terms with sections of the represented family; `representedTerm_at` recovers their actual simultaneous substitution. `accepted_has_represented_term` connects a successful executable check to that same source term and its contextual substitutions.

The boundary has both positive and negative witnesses. `dependent_witness_recovered` recovers a term in a genuinely term-dependent identity family. `beta_conversion_does_not_collapse_sections` exhibits well-typed, beta-convertible source terms with distinct source-retaining sections. `upper_sort_not_formed` rejects `u1` as an ordinary represented type. These results support faithful retention of the stated source syntax; they do not provide identity elimination, cumulative universes, or full native-candidate initiality.

6 MeTTa presentations and certificate experiments

6.1 Ownership of the outer presentation

The outer locally nameless Pattern presentation is located in `Mettapedia/Languages/MeTTa/Experimental/TwoSortPiSigmaId/Pattern/`. Its `Core.lean` defines the authored `twoSortDependent` language definition. This is a MeTTa-shaped presentation of the experiment, not the definition of all MeTTa.

The adjacent `Adapters/` directory contains quotation, presentation-typing bridges, and regular Pattern elaboration. The intrinsic regular calculus does not acquire MeTTa semantics merely because its syntax can be quoted into a Pattern.

6.2 Regular Pattern elaboration

`Adapters/RegularPatternElaboration.lean` elaborates supported Pattern syntax into the constant-free intrinsic fragment under an explicit naming environment and binder policy. `elaborateRegularPattern_reflects` proves quotation agreement for a successful elaboration; `elaborateRegularPattern_quote_complete` recovers quoted constant-free terms under its environment hypotheses. The result is not a parser for arbitrary MeTTa programs.

The diagnostic admission function `elaborateAndCheckRegularPattern` uses this order:

1. elaborate the expected type;
2. prepare its formation status;
3. elaborate the term;
4. check against that expected type.

The Boolean-only path elaborates both expressions before invoking the intrinsic checker. Under the stated good-environment hypothesis, `regularPatternCheckBool_correct` proves equivalence with the independent `RegularPatternPublicChecks` predicate. `RegularPatternAdmission.regularJudgment` extracts regular typing from successful admission. Positive admission and negative rejection are witnessed by `regularPatternTypingDecisionKernel_accepts_positive` and `regularPatternTypingDecisionKernel_rejects_negative`; unknown names and binder collisions have separate rejection theorems.

6.3 Historical closed certificates

`Services/CertificateFragment.lean` retains the smaller `TwoSortSyntaxTerm` grammar and two maps: `toScopedTerm` into scoped syntax and `toClosedPattern` into the shared artifact representation. `toClosedPattern_eq_quoteClosedTm` proves the two quotations agree.

This certificate has a deliberately limited meaning. `TwoSortCertificate` contains a term, an artifact, and their agreement; agreement alone does not establish typing. `CheckedTwoSortCertificate` additionally carries an empty-context `HasType` witness, which is the permissive relation. It is not a `RegularJudgment`. The corresponding checking, bidirectional, and canonical-evaluation services

retain their own stated hypotheses. In particular, transporting permissive typing along a `cdev` reduction neither regularizes that typing nor upgrades the development to a normal form.

The historical identity application to `u0`, with claimed type `u1`, therefore illustrates the permissive service rather than regular polymorphism. Its use of an identity with domain `u1` is precisely the kind of formation omission separated in Section 3. The regularly typed identity at $\Pi(x : u0).u0$ is the appropriate positive example for the regular calculus.

7 Declaration and family experiments

The declaration-aware modules retain their own mathematical scope: `DeclarationEnv.lean`, `DeclarationSpec.lean`, and `DeclarationSemantics.lean` define names, environments, declaration specifications, and declaration-aware typing and reduction. They are not extensions already covered by the declaration-free regular normalization theorem.

`InductiveDecl.lean` implements `checkIndDecl`, a syntactic validator that emits declaration specifications after checking name distinctness, constructor targets, and its authored positivity predicate. `check_unitDecl`, `check_boolDecl`, and `check_natDecl` prove the expected outputs for the pilot declarations. `check_badDupDecl`, `check_badNegDecl`, and `check_badTargetDecl` prove rejection of selected invalid inputs.

`Telescope.lean` provides `telescopePi` and `telescopeLam` using the existing context type. Its `checkParamIndDecl` adds parameter-prefix and target checks; `check_listDecl` is a positive parametric example and `check_badParamMismatch` is a negative one. These are concrete computation theorems. They should not be expanded into a semantic soundness theorem for arbitrary inductive declarations merely from the names “positivity” or “checker.”

In particular, the emitted pilot family specifications use `u0` as a type code in a declaration experiment. This does not change the ground-type interpretation of `u0` in the regular calculus or make those declarations regularly formed. The declaration-aware typing relation and its assumptions must be inspected independently.

`RecursorDecl.lean` retains pilot recursor contracts and selected computation infrastructure. The non-dependent shapes `unitRecType`, `boolRecType`, and `natRecType` are not a general dependent-eliminator generator. A future extension would need explicit dependent motives, the intended constructor cases and induction hypotheses, well-formed generated types, and preservation for the generated computation rules. The conditional design discipline is still useful: declaration lookup unfolding and eliminator computation are distinct, and should not be conflated. It is not a claim that extending this old experiment is the selected path for Prime.

8 The verified runtime frontier

The historical architecture sought a shared elaborated source with a proof-oriented target and a runtime target. The verified overlap relevant here is more restricted.

`Services/RuntimeFrontier.lean` classifies the three authored Pattern rewrites against the particular direct MORK source-rule bridge:

Rule	Left-hand side	Right-hand side
BetaPi	application constructor	explicit substitution
BetaSigmaFst	projection constructor	matched variable
BetaSigmaSnd	projection constructor	matched variable

That bridge requires a free-variable left-hand side and a translatable right-hand side. None of these three left-hand sides has the required form; BetaPi also fails right-hand-side translatability. The summary theorem is `no_twoSortDependent_rewrite_fits_direct_runtimeExec0_source_bridge`. This is an obstruction to that bridge’s hypotheses, not an impossibility theorem for every operational implementation.

The positive path is quotation: `closedTwoSort_overlap_via_abc` sends the specified closed operational step to the quoted profile-theory step. `closedTwoSort_overlap_via_abc_to_wm` transports it to a world-model strength obligation under the supplied interface and side condition. It does not execute the term in MORK or MM2.

`certifyTwoSortSyntax_overlapClass` assigns `artifactOnly`, and `twoSortClosedSyntax_overlap_is_not_directExec` proves the negative classification for that certificate constructor. By contrast, separate runtime developments prove direct execution for specific MeTTa rule/query/effect fragments, such as `pettaEvalStep_schedulerFires` in `Mettapedia/Languages/ProcessCalculi/MORK/DirectExecInterface.lean`. Those runtime results do not turn the two-sort proof experiment into a dual-target language.

A shared artifact is a common representation. It is not by itself a simulation, preservation theorem, or proof that a runtime and a proof calculus compute the same result. Any stronger overlap statement must name the source fragment, both operational relations, and the connecting theorem.

9 Relation to a candidate Prime theory

The cumulative universe-tower development is separately located at `Mettapedia/Languages/MeTTa/TypeTheory/CumulativeTower.lean` and its associated directory. Its umbrella explicitly presents a candidate investigated by Prime, not Prime’s final or exclusive type theory. This paper does not select that candidate, transplant the two-sort normalization theorem to it, or infer a hierarchy from the two fixed heads.

Useful transfers are specific mathematical obligations:

1. Define a translation on the intended fragment, respecting scope and simultaneous substitution.
2. Prove which typing relation is preserved, with context and type-formation hypotheses explicit.
3. State and prove the conversion or reduction correspondence needed by the consumer. Preservation alone is not reflection or conservativity.
4. Re-establish normalization or checking properties for the actual target judgment when the translation theorem does not imply them.

For example, the identity term and dependent identity family above are positive tests for a proposed fragment interpretation; the forbidden `u1`-domain and untyped Ω are negative controls for its claims. Preserving a scoped encoding while accepting an invalid regular judgment would not constitute a typing-preserving interpretation.

The correct organizational conclusion is therefore modest and durable: retain the two-sort calculus as named mathematics, retain the MeTTa-specific experiments as adapters, and evaluate candidate Prime theories through their own rules and theorems.

10 Source and claim index

The principal source anchors are listed below. File names refer to the reorganized source tree, not the historical mixed kernel directory. No global line count or blanket “zero axioms” claim substitutes for checking the imported dependencies and hypotheses of an individual theorem.

Intrinsic module	Principal claim or object
<code>Syntax.lean</code>	<code>ScopedTerm</code> ; fixed heads and scoped grammar.
<code>Confluence.lean</code>	<code>redStar_confluence</code> , <code>church_rosser_conv</code> .
<code>Permissive/Typing.lean</code>	The historical <code>HasType</code> , not regular typing.
<code>Regular/Typing.lean</code>	<code>RegularJudgment</code> ; formation premises and negative controls.
<code>Regular/Structural.lean</code>	Regular substitution and type presupposition.
<code>Regular/DefEq.lean</code>	<code>regularDefEq_not_complete</code> : one development is insufficient.
<code>Regular/Fundamental.lean</code>	<code>RegularJudgment.subject_accessible</code> .
<code>Regular/Normalization.lean</code>	<code>regularDecidedConversion_correct</code> on its proved domain.
<code>Regular/BidirectionalCompleteness.lean</code>	<code>regularCheckBool_correct</code> against <code>RegularPublicChecks</code> .
<code>Regular/SyntacticCwf.lean</code>	<code>cwfWithTerminal</code> ; raw-syntax equality, separate conversion.
<code>Regular/PresheafSemantics.lean</code>	<code>accepted_has_represented_term</code> ; <code>beta_conversion_does_not_collapse_sections</code> .

The MeTTa-specific anchors are `Experimental/TwoSortPiSigmaId/Pattern/Core.lean`, `Experimental/TwoSortPiSigmaId/Adapters/RegularPatternElaboration.lean`, and the `Services/` modules described above, all below `Mettapedia/Languages/MeTTa/`. The generic categorical machinery remains in `Mettapedia/TypeTheory/CwfYoneda.lean` and `Mettapedia/TypeTheory/CwfYonedaCoherence.lean`.

11 Conclusion

The development supplies a substantive, fixed two-sort $\Pi/\Sigma/\text{Id}$ fragment with a separately identified regular judgment and verified normalization and checking boundaries. Its limitations—no J , no universe hierarchy, and no automatic transfer to declaration experiments or candidate Prime—are part of its precise mathematical identity, not reasons to discard the proofs.

The historical MeTTa presentation explains how the calculus acquired its earlier names. It does not determine what Prime must be.